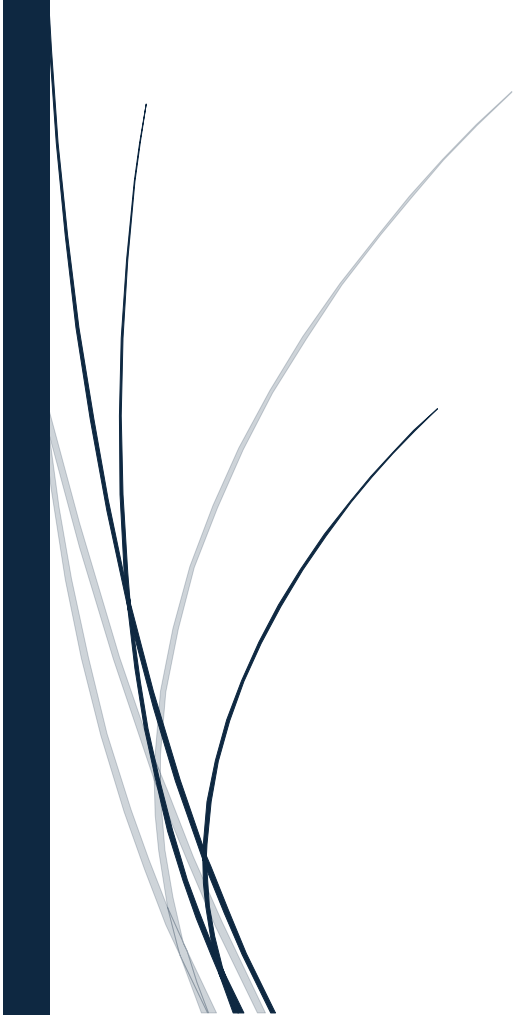
CSC 212 Project

Ride-Sharing System



King Saud University | Department of Computer Science

Ride-Sharing System

Data structure Project — Phase 1

Course Information:

Course Code: CSC 220

Course Name: Data structure

Prepared By:

Student name	Student ID
Abdulrahman Alamer	445102743
Refaei Al-Otaibi	446103528
Ahmed AL-Bakheet	446105300

Submitted To: Dr. Faisal Abdullah Al-Atawi

Submission Date: 31 October 2026 (Saturday)

Academic Year: 2026/2027

System Design: Ride-Sharing System

1. Division of Work

Table 1: Class Implementation

Title	Description	Classes	Assigned Student
Rider Management + Core DS	Implementation of rider-related classes, rider operations, and custom data structures.	Person, Rider, Driver RiderList, List, Node, LinkedList	Abdulrahman Alamer
Driver Management + Time	Implementation of driver-related classes, driver operations, and date-time functionality.	DriverList, DateTime	Refaei Al-Otaibi
Ride Management	Implementation of ride-related classes, including private and shared rides and ride storage.	Ride, PrivateRide, SharedRide, RideList	Ahmed AL-Bakheet

Table 2: System Management (System Class)

Title	Description	Methods	Assigned Student
CSV Management	Loading and processing rider, driver, and ride records from CSV files.	loadRidersFromCSV(), loadDriversFromCSV(), loadRidesFromCSV()	Abdulrahman Alamer
Rider Management	Managing rider records, including addition, searching, retrieval, and removal.	addRider(), searchRiderById(), searchRiderByEmail(), searchRidersByName(), searchRidersByHomeCity(), getAllRiders(), removeRider()	Refaei Al-Otaibi
Driver Management	Managing driver records, including addition, searching, retrieval, and removal.	addDriver(), searchDriverById(), searchDriverByVehiclePlate(), searchDriversByVehicleType(), getAllDrivers(), removeDriver()	Refaei Al-Otaibi
Ride Management	Scheduling private and shared rides, enforcing scheduling constraints, and searching for rides and participants.	schedulePrivateRide(), scheduleSharedRide(), searchRidesByPickupLocation(), searchRidesByRiderName(), getSharedRideParticipants(), getAllRidesAlphabetically()	Refaei Al-Otaibi

Table 3: Main Class

Title	Description	Classes	Assigned Student
Command-Line Interface	Implementation of the main menu, user input handling, input validation, and output formatting.	Main	Ahmed AL-Bakheet

2. Time Complexity Analysis (Worst-Case Frequency Analysis)

- LinkedList Class

1. LinkedList ()

Statement	S/E	Frequency	Total
public LinkedList ()	0	–	0
{	0	–	0
head = current = null;	1	1	1
size = 0;	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O(1)**

2. empty ()

Statement	S/E	Frequency	Total
public boolean empty ()	0	–	0
{	0	–	0
return head == null;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

3. last ()

Statement	S/E	Frequency	Total
public boolean last ()	0	–	0
{	0	–	0
return current.next == null;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

4. first ()

Statement	S/E	Frequency	Total
public boolean first ()	0	–	0
{	0	–	0
return current.previous == null;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

5. full ()

Statement	S/E	Frequency	Total
public boolean full ()	0	–	0
{	0	–	0
return false;	1	1	1

}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. findFirst ()

Statement	S/E	Frequency	Total
public void findFirst ()	0	–	0
{	0	–	0
current = head;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

7. findPrevious ()

Statement	S/E	Frequency	Total
public void findPrevious ()	0	–	0
{	0	–	0
current = current.previous;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. findNext ()

Statement	S/E	Frequency	Total
public void findNext ()	0	–	0
{	0	–	0
current = current.next;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

9. retrieve ()

Statement	S/E	Frequency	Total
public T retrieve ()	0	–	0
{	0	–	0
return current.data;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

10. update (T val)

Statement	S/E	Frequency	Total
public void update (T val)	0	–	0
{	0	–	0
current.data = val;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

11. insert (T val)

Statement	S/E	Frequency	Total
public void insert (T val)	0	–	0
{	0	–	0
Node<T> tmp = new Node<T>(val);	1	1	1
if(empty())	1	1	1
{	0	–	0
current = head = tmp;	1	0	0
}	0	–	0
else	0	–	0
{	0	–	0
tmp.next = current.next;	1	1	1
tmp.previous = current;	1	1	1
if(current.next != null)	1	1	1
current.next.previous = tmp;	1	1	1
current.next = tmp;	1	1	1
current = tmp;	1	1	1
}	0	–	0
size++;	1	1	1
}	0	–	0
Total	–	–	**9**
Big-O	–	–	**O (1)**

12. insertFirst (T val)

Statement	S/E	Frequency	Total
public void insertFirst (T val)	0	–	0
{	0	–	0
Node<T> element = new Node<>(val);	1	1	1
if (empty())	1	1	1
{	0	–	0
head = current = element;	1	0	0
} else {	0	–	0
element.next = head;	1	1	1
head.previous = element;	1	1	1
head = element;	1	1	1
}	0	–	0
current = head;	1	1	1
size++;	1	1	1
}	0	–	0
Total	–	–	**7**
Big-O	–	–	**O (1)**

13. remove ()

Statement	S/E	Frequency	Total
public void remove ()	0	–	0
{	0	–	0
if (empty() || current == null)	1	1	1

{	0	–	0
return;	1	0	0
}	0	–	0
if(current == head)	1	1	1
{	0	–	0
head = head.next;	1	0	0
if(head != null) head.previous = null;	1	0	0
} else {	0	–	0
current.previous.next = current.next;	1	1	1
if(current.next != null) current.next.previous = current.previous;	1	1	1
}	0	–	0
if (current.next == null) current = head;	1	1	1
else current = current.next;	1	0	0
size--;	1	1	1
}	0	–	0
Total	–	–	**6**
Big-O	–	–	**O (1)**

14. size ()

Statement	S/E	Frequency	Total
public int size ()	0	–	0
{	0	–	0
return size;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- Person Class:

1. Person (int id, String name, String phoneNumber)

Statement	S/E	Frequency	Total
public Person (int id, String name, String phoneNumber) throws IllegalArgumentException	0	–	0
{	0	–	0
this.id = id;	1	1	1
setName(name);	1	1	1
setPhoneNumber(phoneNumber);	1	1	1
rideHistory = new LinkedList<IRide>();	1	1	1
}	0	–	0
Total	–	–	**4**
Big-O			**O (1)**

2. setName (String name)

Statement	S/E	Frequency	Total
public void setName (String name)	0	–	0
{	0	–	0
this.name = name;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

3. setPhoneNumber (String phoneNumber)

Statement	S/E	Frequency	Total
public void setPhoneNumber (String phoneNumber) throws IllegalArgumentException	0	–	0
{	0	–	0
if (phoneNumber == null || !phoneNumber.matches("\\d{10}"))	1	1	1
{	0	–	0
throw new IllegalArgumentException("Phone number must contain exactly 10 digits.");	1	0	0
}	0	–	0
this.phoneNumber = phoneNumber;	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O (1)**

4. getId ()

Statement	S/E	Frequency	Total
public int getId ()	0	–	0
{	0	–	0
return id;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

5. getName ()

Statement	S/E	Frequency	Total
public String getName ()	0	–	0
{	0	–	0
return name;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. getPhoneNumber ()

Statement	S/E	Frequency	Total
public String getPhoneNumber()	0	–	0
{	0	–	0
return phoneNumber;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

7. getRideHistory ()

Statement	S/E	Frequency	Total
public LinkedList<IRide> getRideHistory()	0	–	0
{	0	–	0
return rideHistory;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. toString ()

Statement	S/E	Frequency	Total
public String toString()	0	–	0
{	0	–	0
return "ID: " + id + ", Name: " + name + ", Phone Number: " + phoneNumber;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- Rider Class:

1. Rider (int id, String name, String phoneNumber, String email, String homeCity)

Statement	S/E	Frequency	Total
public Rider (int id, String name, String phoneNumber, String email, String homeCity) throws IllegalArgumentException	0	–	0
{	0	–	0
super(id, name, phoneNumber);	1	1	1
setEmail(email);	1	1	1
setHomeCity(homeCity);	1	1	1
}	0	–	0
Total	–	–	**3**
Big-O	–	–	**O (1)**

2. setEmail (String email)

Statement	S/E	Frequency	Total
public void setEmail (String email)	0	–	0
{	0	–	0
if (email == null) throw new IllegalArgumentException ("Email can't be null");	1	1	1
this.email = email;	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O (1)**

3. setHomeCity(String homeCity)

Statement	S/E	Frequency	Total
public void setHomeCity (String homeCity)	0	–	0
{	0	–	0
this.homeCity = homeCity;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O(1)**

4. getEmail ()

Statement	S/E	Frequency	Total
public String getEmail ()	0	–	0
{	0	–	0
return email;	1	1	1
}	0	–	0
Total			**1**
Big-O			**O (1)**

5. getHomeCity ()

Statement	S/E	Frequency	Total
public String getHomeCity ()	0	–	0
{	0	–	0
return homeCity;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. toString ()

Statement	S/E	Frequency	Total
public String toString ()	0	–	0
{	0	–	0
return super.toString() + ", Email: " + email + ", Home City: " + homeCity;	1	1	1
}	0	–	0

Total			**1**
Big-O			**O (1)**

7. compareTo (IRider other)

Statement	S/E	Frequency	Total
public int compareTo(IRider other)	0	–	0
{	0	–	0
if (this.getId() < other.getId())	1	1	1
{	0	–	0
return -1;	1	0	0
} else if (this.getId() > other.getId()) {	1	1	1
return 1;	1	0	0
} else {	0	–	0
return 0;	1	1	1
}	0	–	0
}	0	–	0
Total			**3**
Big-O			**O (1)**

- Driver Class:

1. Driver (int id, String name, String phoneNumber, String vehiclePlate, VehicleType vehicleType)

Statement	S/E	Frequency	Total
public Driver (int id, String name, String phoneNumber, String vehiclePlate, VehicleType vehicleType) throws IllegalArgumentException {	0	–	0
super(id, name, phoneNumber);	1	1	1
setVehiclePlate(vehiclePlate);	1	1	1
setVehicleType(vehicleType);	1	1	1
}	0	–	0
Total	–	–	**3**
Big-O			**O (1)**

2. setVehiclePlate (String vehiclePlate)

Statement	S/E	Frequency	Total
public void setVehiclePlate (String vehiclePlate) throws IllegalArgumentException {	0	–	0
if (vehiclePlate == null || !vehiclePlate.matches("[A-Z]{3}\\d{4}")) {	1	1	1
throw new IllegalArgumentException("Vehicle plate must be exactly 3 uppercase letters followed by 4 digits.");	1	0	0

}	0	–	0
this.vehiclePlate = vehiclePlate;	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O(1)**

3. setVehicleType (VehicleType vehicleType)

Statement	S/E	Frequency	Total
public void setVehicleType(VehicleType vehicleType) throws IllegalArgumentException {	0	–	0
if (vehicleType == null) throw new IllegalArgumentException("Vehicle type can't be null");	1	1	1
this.vehicleType = vehicleType;	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O			**O (1)**

4. getVehiclePlate ()

Statement	S/E	Frequency	Total
public String getVehiclePlate() {	0	–	0
return vehiclePlate;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O			**O (1)**

5. getVehicleType ()

Statement	S/E	Frequency	Total
public VehicleType getVehicleType() {	0	–	0
return vehicleType;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O 1)**

6. toString ()

Statement	S/E	Frequency	Total
public String toString() {	0	–	0
return super.toString() + ", Vehicle Plate: " + vehiclePlate + ", Vehicle Type: " + vehicleType;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O(1)**

7. compareTo (IDriver other)

Statement	S/E	Frequency	Total
public int compareTo(IDriver other) {	0	–	0
return Integer.compare(this.getId(), other.getId());	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- DateTime Class:

1. DateTime (int year, int month, int day, int hour, int minute)

Statement	S/E	Frequency	Total
public DateTime (int year, int month, int day, int hour, int minute) throws IllegalArgumentException {	0	–	0
try {	0	–	0
LocalDateTime.of(year, month, day, hour, minute);	1	1	1
} catch (DateTimeException e) {	0	–	0
throw new IllegalArgumentException("Invalid date or time.", e);	1	0	0
}	0	–	0
this.year = year;	1	1	1
this.month = month;	1	1	1
this.day = day;	1	1	1
this.hour = hour;	1	1	1
this.minute = minute;	1	1	1
}	0	–	0
Total	–	–	**6**
Big-O	–	–	**O (1)**

2. format ()

Statement	S/E	Frequency	Total
public String format () {	0	–	0
return String.format("%02d/%02d/%04d %02d:%02d", month, day, year, hour, minute);	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

3. compareTo (IDateTime other)

Statement	S/E	Frequency	Total
public int compareTo(IDateTime other) {	0	–	0
if (this.year != other.getYear())	1	1	1
return Integer.compare(this.year, other.getYear());	1	0	0
if (this.month != other.getMonth())	1	1	1
return Integer.compare(this.month, other.getMonth());	1	0	0
if (this.day != other.getDay())	1	1	1
return Integer.compare(this.day, other.getDay());	1	0	0
if (this.hour != other.getHour())	1	1	1
return Integer.compare(this.hour, other.getHour());	1	0	0
return Integer.compare(this.minute, other.getMinute());	1	1	1
}	0	–	0
Total	–	–	**5**
Big-O	–	–	**O (1)**

4. getYear ()

Statement	S/E	Frequency	Total
public int getYear() {	0	–	0
return year;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

5. getMonth ()

Statement	S/E	Frequency	Total
public int getMonth() {	0	–	0
return month;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. getDay ()

Statement	S/E	Frequency	Total
public int getDay() {	0	–	0
return day;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

7. getHour ()

Statement	S/E	Frequency	Total
public int getHour() {	0	–	0
return hour;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. getMinute ()

Statement	S/E	Frequency	Total
public int getMinute() {	0	–	0
return minute;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- Ride Class:

1. Ride (int rideId, IDateTime pickupTime, IDateTime dropoffTime, String pickupLocation, String dropoffLocation, IDriver driver)

Statement	S/E	Frequency	Total
public Ride(int rideId, IDateTime pickupTime, IDateTime dropoffTime, String pickupLocation, String dropoffLocation, IDriver driver) {	0	–	0
if (pickupTime == null || dropoffTime == null)	1	1	1
throw new IllegalArgumentException("Pickup and drop-off times cannot be null.");	1	1	1
if (pickupTime.compareTo(dropoffTime) >= 0)	1	1	1
throw new IllegalArgumentException("Drop-off time must be later than pickup time.");	1	1	1
this.rideId = rideId;	1	1	1
this.pickupTime = pickupTime;	1	1	1
this.dropoffTime = dropoffTime;	1	1	1
setDropoffLocation(dropoffLocation);	1	1	1
setPickupLocation(pickupLocation);	1	1	1
setDriver(driver);	1	1	1
}	0	–	0
Total	–	–	**10**
Big-O	–	–	**O (1)**

2. compareTo (IRide other)

Statement	S/E	Frequency	Total
public int compareTo(IRide other) {	0	–	0
return this.pickupLocation.compareToIgnoreCase(other.getPickupLocation());	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

3. setPickupLocation (String pickupLocation)

Statement	S/E	Frequency	Total
public void setPickupLocation(String pickupLocation) {	0	–	0
this.pickupLocation = pickupLocation;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

4. setDropoffLocation (String dropoffLocation)

Statement	S/E	Frequency	Total
public void setDropoffLocation(String dropoffLocation) {	0	–	0
this.dropoffLocation = dropoffLocation;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

5. setDriver (IDriver driver)

Statement	S/E	Frequency	Total
public void setDriver(IDriver driver) {	0	–	0
this.driver = driver;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. getRideId ()

Statement	S/E	Frequency	Total
public int getRideId() {	0	–	0
return rideId;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

7. getPickupTime ()

Statement	S/E	Frequency	Total
public IDateTime getPickupTime() {	0	–	0
return pickupTime;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. getDropoffTime ()

Statement	S/E	Frequency	Total
public IDateTime getDropoffTime() {	0	–	0
return dropoffTime;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

9. getPickupLocation ()

Statement	S/E	Frequency	Total
public String getPickupLocation() {	0	–	0
return pickupLocation;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

10. getDropoffLocation ()

Statement	S/E	Frequency	Total
public String getDropoffLocation() {	0	–	0
return dropoffLocation;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

11. getDriver ()

Statement	S/E	Frequency	Total
public IDriver getDriver() {	0	–	0
return driver;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

12. toString ()

Statement	S/E	Frequency	Total
public String toString() {	0	–	0

return String.format("Ride ID: %d, Driver: %d - %s, Pickup: %s, Pickup Time: %s, Drop-off: %s, Drop-off Time: %s",	1	1	1
rideId, driver.getId(), driver.getName(), pickupLocation, pickupTime.format(), dropoffLocation, dropoffTime.format());	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O (1)**

- PrivateRide Class:

1. PrivateRide (int rideId, IDateTime pickupTime, IDateTime dropoffTime, String pickupLocation, String dropoffLocation, IDriver driver, IRider rider)

Statement	S/E	Frequency	Total
public PrivateRide(int rideId, IDateTime pickupTime, IDateTime dropoffTime, String pickupLocation, String dropoffLocation, IDriver driver, IRider rider)	0	–	0
{	0	–	0
super(rideId, pickupTime, dropoffTime, pickupLocation, dropoffLocation, driver);	1	1	1
setRider(rider);	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O (1)**

2. hasRider (int riderId)

Statement	S/E	Frequency	Total
public boolean hasRider(int riderId)	0	–	0
{	0	–	0
return rider.getId() == riderId;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

3. getRider()

Statement	S/E	Frequency	Total
public IRider getRider()	0	–	0
{	0	–	0
return rider;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

4. setRider (IRider rider)

Statement	S/E	Frequency	Total
public void setRider(IRider rider)	0	–	0
{	0	–	0
if (rider == null)	1	1	1
throw new IllegalArgumentException("Rider cannot be null.");	1	1	1
this.rider = rider;	1	1	1
}	0	–	0
Total	–	–	**3**
Big-O	–	–	**O (1)**

5. toString ()

Statement	S/E	Frequency	Total
public String toString()	0	–	0
{	0	–	0
return String.format("Private, %s, Rider: %s (ID %d)", super.toString(), rider.getName(), rider.getId());	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- SharedRide Class:

1. SharedRide (...)

Statement	S/E	Frequency	Total
public SharedRide(int rideId, IDateTime pickupTime, IDateTime dropoffTime, String pickupLocation, String dropoffLocation, IDriver driver)	0	–	0
{	0	–	0
super(rideId, pickupTime, dropoffTime, pickupLocation, dropoffLocation, driver);	1	1	1
participants = new LinkedList<IRider>();	1	1	1
}	0	–	0
Total	–	–	**2**
Big-O	–	–	**O (1)**

2. hasRider (int riderId)

Statement	S/E	Frequency	Total
public boolean hasRider(int riderId)	0	–	0
{	0	–	0

if (participants.empty())	1	1	1
return false;	1	0	0
participants.findFirst();	1	1	1
while (true)	1	n	n
{	0	–	0
if (participants.retrieve().getId() == riderId)	1	n	n
return true;	1	0	0
if (participants.last())	1	n	n
break;	1	1	1
participants.findNext();	1	n-1	n-1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**4n+3**
Big-O	–	–	**O (n)**

3. getParticipants ()

Statement	S/E	Frequency	Total
public LinkedList<IRider> getParticipants()	0	–	0
{	0	–	0
return participants;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

4. addParticipant (IRider rider)

Statement	S/E	Frequency	Total
public boolean addParticipant(IRider rider)	0	–	0
{	0	–	0
if (rider == null || hasRider(rider.getId()))	1	1	1
return false;	1	0	0
participants.insert(rider);	1	1	1
return true;	1	1	1
}	0	–	0
Total	–	–	**O(n) [hasRider]**
Big-O	–	–	**O (n)**

5. removeParticipantById (int riderId)

Statement	S/E	Frequency	Total
public boolean removeParticipantById(int riderId)	0	–	0
{	0	–	0
if (participants.empty())	1	1	1
return false;	1	0	0
participants.findFirst();	1	1	1

while (true)	1	n	n
{	0	–	0
if (participants.retrieve().getId() == riderId)	1	n	n
{	0	–	0
participants.remove();	1	0	0
return true;	1	0	0
}	0	–	0
if (participants.last())	1	n	n
break;	1	1	1
participants.findNext();	1	n-1	n-1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**4n+3**
Big-O	–	–	**O (n)**

6. isEmpty ()

Statement	S/E	Frequency	Total
public boolean isEmpty()	0	–	0
{	0	–	0
return participants.empty();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

7. toString ()

Statement	S/E	Frequency	Total
public String toString()	0	–	0
{	0	–	0
if (participants.empty())	1	1	1
return String.format("%s, Riders: None", super.toString());	1	0	0
StringBuilder riders = new StringBuilder("Riders: ");	1	1	1
participants.findFirst();	1	1	1
while (true)	1	n	n
{	0	–	0
riders.append(String.format("%s (ID: %d)", participants.retrieve().getName(), participants.retrieve().getId()));	1	n	n
if (participants.last())	1	n	n
break;	1	1	1
riders.append(" | ");	1	n-1	n-1
participants.findNext();	1	n-1	n-1
}	0	–	0
return String.format("Shared, %s, %s", super.toString(), riders);	1	1	1
}	0	–	0

Total	–	–	**5n+2**
Big-O	–	–	**O (n)**

- RiderList Class:

1. RiderList ()

Statement	S/E	Frequency	Total
public RiderList() {	0	–	0
riders = new LinkedList<IRider>();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

2. boolean add (IRider rider)

Statement	S/E	Frequency	Total
public boolean add (IRider rider) {	0	–	0
if (rider == null || findById(rider.getId()) != null || findByEmail(rider.getEmail()) != null)	1	1	1
return false;	1	1	1
if (riders.empty()) {	1	1	1
riders.insert(rider);	1	1	1
return true;	1	1	1
}	0	–	0
riders.findFirst();	1	1	1
if (riders.retrieve().compareTo(rider) > 0) {	1	1	1
riders.insertFirst(rider);	1	1	1
return true;	1	1	1
}	0	–	0
while (!riders.last()) {	1	n	n
riders.findNext();	1	n−1	n−1
if (riders.retrieve().compareTo(rider) > 0) {	1	n−1	n−1
riders.findPrevious();	1	1	1
riders.insert(rider);	1	1	1
return true;	1	0	0
}	0	–	0
}	0	–	0
riders.insert(rider);	1	1	1
return true;	1	1	1
}	0	–	0
Total	–	–	**3n + 11**
Big-O	–	–	**O (n)**

3. IRider findById (int riderId)

Statement	S/E	Frequency	Total
public IRider findById(int riderId) {	0	–	0
if (riders.empty()) return null;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getId() == riderId)	1	n–1	n–1
return riders.retrieve();	1	0	0
riders.findNext();	1	n–1	n–1
}	0	–	0
return riders.retrieve().getId() == riderId ? riders.retrieve() : null;	1	1	1
}	0	–	0
Total	–	–	**3n + 1**
Big-O	–	–	**O(n)**

4. LinkedList<IRider> findByName (String fullName)

Statement	S/E	Frequency	Total
public LinkedList<IRider> findByName(String fullName) {	0	–	0
LinkedList<IRider> result = new LinkedList<IRider>();	1	1	1
if (riders.empty())	1	1	1
return result;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
result.insert(riders.retrieve());	1	1	1
}	0	–	0
riders.findNext();	1	n–1	n–1
}	0	–	0
if (riders.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
result.insert(riders.retrieve());	1	1	1
}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O (n)**

5. IRider findByEmail (String email)

Statement	S/E	Frequency	Total
public IRider findByEmail(String email) {	0	–	0
if (riders.empty())	1	1	1
return null;	1	0	0
riders.findFirst();	1	1	1

while (!riders.last()) {	1	n	n
if (riders.retrieve().getEmail().equalsIgnoreCase(email))	1	n–1	n–1
return riders.retrieve();	1	0	0
riders.findNext();	1	n–1	n–1
}	0	–	0
return riders.retrieve().getEmail().equalsIgnoreCase(email) ? riders.retrieve() : null;	1	1	1
}	0	–	0
Total	–	–	**3n + 1**
Big-O	–	–	**O (n)**

6. LinkedList<IRider> findByHomeCity (String homeCity)

Statement	S/E	Frequency	Total
public LinkedList<IRider> findByHomeCity(String homeCity) {	0	–	0
LinkedList<IRider> result = new LinkedList<IRider>();	1	1	1
if (riders.empty())	1	1	1
return result;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getHomeCity().equalsIgnoreCase(homeCity)) {	1	n–1	n–1
result.insert(riders.retrieve());	1	1	1
}	0	–	0
riders.findNext();	1	n–1	n–1
}	0	–	0
if (riders.retrieve().getHomeCity().equalsIgnoreCase(homeCity)) {	1	n–1	n–1
result.insert(riders.retrieve());	1	1	1
}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O (n)**

7. LinkedList<IRider> getAll ()

Statement	S/E	Frequency	Total
public LinkedList<IRider> getAll() {	0	–	0
return riders;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. boolean removeById (int riderId)

Statement	S/E	Frequency	Total
public boolean removeById(int riderId) {	0	–	0
if (riders.empty())	1	1	1
return false;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getId() == riderId) {	1	n–1	n–1
riders.remove();	1	1	1
return true;	1	0	0
}	0	–	0
riders.findNext();	1	n–1	n–1
}	0	–	0
if (riders.retrieve().getId() == riderId) {	1	n–1	n–1
riders.remove();	1	1	1
return true;	1	0	0
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**4n + 3**
Big-O	–	–	**O (n)**

9. boolean removeByEmail (String email)

Statement	S/E	Frequency	Total
public boolean removeByEmail(String email) {	0	–	0
if (riders.empty())	1	1	1
return false;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getEmail().equalsIgnoreCase(email)) {	1	n–1	n–1
riders.remove();	1	1	1
return true;	1	0	0
}	0	–	0
riders.findNext();	1	n–1	n–1
}	0	–	0
if (riders.retrieve().getEmail().equalsIgnoreCase(email)) {	1	n–1	n–1
riders.remove();	1	1	1
return true;	1	0	0
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**4n + 3**
Big-O	–	–	**O (n)**

10. int removeByName (String fullName)

Statement	S/E	Frequency	Total
public int removeByName(String fullName) {	0	–	0
if (riders.empty())	1	1	1
return 0;	1	0	0
int count = 0;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
riders.remove();	1	0	0
count++;	1	0	0
} else {	1	1	1
riders.findNext();	1	n–1	n–1
}	0	–	0
}	0	–	0
if (riders.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
riders.remove();	1	0	0
count++;	1	0	0
}	0	–	0
return count;	1	1	1
}	0	–	0
Total	–	–	**4n + 2**
Big-O	–	–	**O (n)**

11. int removeByHomeCity (String homeCity)

Statement	S/E	Frequency	Total
public int removeByHomeCity(String homeCity) {	0	–	0
if (riders.empty())	1	1	1
return 0;	1	0	0
int count = 0;	1	1	1
riders.findFirst();	1	1	1
while (!riders.last()) {	1	n	n
if (riders.retrieve().getHomeCity().equalsIgnoreCase(homeCity)) {	1	n–1	n–1
riders.remove();	1	0	0
count++;	1	0	0
} else {	1	1	1
riders.findNext();	1	n–1	n–1
}	0	–	0
}	0	–	0
if (riders.retrieve().getHomeCity().equalsIgnoreCase(homeCity)) {	1	n–1	n–1
riders.remove();	1	0	0

count++;	1	0	0
}	0	–	0
return count;	1	1	1
}	0	–	0
Total	–	–	**4n + 2**
Big-O	–	–	**O (n)**

12. int size ()

Statement	S/E	Frequency	Total
public int size() {	0	–	0
return riders.size();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O(1)**

- DriverList

1. DriverList ()

Statement	S/E	Frequency	Total
public DriverList() {	0	–	0
drivers = new LinkedList<IDriver>();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

2. boolean add (IDriver driver)

Statement	S/E	Frequency	Total
public boolean add (IDriver driver) {	0	–	0
if (driver == null || findById(driver.getId()) != null || findByVehiclePlate(driver.getVehiclePlate()) != null)	1	1	1
return false;	1	0	0
if (drivers.empty()) {	1	1	1
drivers.insert(driver);	1	1	1
return true;	1	1	1
}	0	–	0
drivers.findFirst();	1	1	1
if (drivers.retrieve().compareTo(driver) > 0) {	1	1	1
drivers.insertFirst(driver);	1	1	1
return true;	1	1	1
}	0	–	0
while (!drivers.last()) {	1	n	n
drivers.findNext();	1	n–1	n–1
if (drivers.retrieve().compareTo(driver) > 0) {	1	n–1	n–1
drivers.findPrevious();	1	1	1
drivers.insert(driver);	1	1	1

return true;	1	0	0
}	0	–	0
}	0	–	0
drivers.insert(driver);	1	1	1
return true;	1	1	1
}	0	–	0
Total	–	–	**3n + 10**
Big-O	–	–	**O(n)**

3. IDriver findById (int driverId)

Statement	S/E	Frequency	Total
public IDriver findById(int driverId) {	0	–	0
if (drivers.empty()) return null;	1	1	1
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getId() == driverId)	1	n–1	n–1
return drivers.retrieve();	1	0	0
drivers.findNext();	1	n–1	n–1
}	0	–	0
if (drivers.retrieve().getId() == driverId)	1	1	1
return drivers.retrieve();	1	0	0
else	0	–	0
return null;	1	1	1
}	0	–	0
Total	–	–	**3n + 2**
Big-O	–	–	**O (n)**

4. LinkedList<IDriver> findByName (String fullName)

Statement	S/E	Frequency	Total
public LinkedList<IDriver> findByName (String fullName) {	0	–	0
LinkedList<IDriver> result = new LinkedList<IDriver>();	1	1	1
if (drivers.empty())	1	1	1
return result;	1	1	1
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
result.insert(drivers.retrieve());	1	n–1	n–1
}	0	–	0
drivers.findNext();	1	n–1	n–1
}	0	–	0
if (drivers.retrieve().getName().equalsIgnoreCase(fullName)) {	1	1	1
result.insert(drivers.retrieve());	1	1	1

}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O (n)**

5. IDriver findByVehiclePlate (String vehiclePlate)

Statement	S/E	Frequency	Total
public IDriver findByVehiclePlate(String vehiclePlate) {	0	–	0
if (drivers.empty())	1	1	1
return null;	1	0	0
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getVehiclePlate().equals(vehiclePlate))	1	n–1	n–1
return drivers.retrieve();	1	0	0
drivers.findNext();	1	n–1	n–1
}	0	–	0
if (drivers.retrieve().getVehiclePlate().equals(vehiclePlate))	1	1	1
return drivers.retrieve();	1	0	0
else	0	–	0
return null;	1	1	1
}	0	–	0
Total	–	–	**3n + 2**
Big-O	–	–	**O (n)**

6. LinkedList<IDriver> findByVehicleType (VehicleType vehicleType)

Statement	S/E	Frequency	Total
public LinkedList<IDriver> findByVehicleType(VehicleType vehicleType) {	0	–	0
LinkedList<IDriver> result = new LinkedList<IDriver>();	1	1	1
if (drivers.empty())	1	1	1
return result;	1	1	1
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getVehicleType() == vehicleType) {	1	n–1	n–1
result.insert(drivers.retrieve());	1	n–1	n–1
}	0	–	0
drivers.findNext();	1	n–1	n–1
}	0	–	0
if (drivers.retrieve().getVehicleType() == vehicleType) {	1	1	1
result.insert(drivers.retrieve());	1	1	1
}	0	–	0

return result;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O (n)**

7. LinkedList<IDriver> getAll()

Statement	S/E	Frequency	Total
public LinkedList<IDriver> getAll() {	0	–	0
return drivers;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

8. boolean removeById(int driverId)

Statement	S/E	Frequency	Total
public boolean removeById(int driverId) {	0	–	0
if (drivers.empty())	1	1	1
return false;	1	0	0
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getId() == driverId) {	1	n−1	n−1
drivers.remove();	1	0	0
return true;	1	0	0
}	0	–	0
drivers.findNext();	1	n−1	n−1
}	0	–	0
if (drivers.retrieve().getId() == driverId) {	1	1	1
drivers.remove();	1	1	1
return true;	1	1	1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**3n + 4**
Big-O	–	–	**O (n)**

9. boolean removeByVehiclePlate (String vehiclePlate)

Statement	S/E	Frequency	Total
public boolean removeByVehiclePlate(String vehiclePlate) {	0	–	0
if (drivers.empty())	1	1	1
return false;	1	0	0
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getVehiclePlate().equals(vehiclePlate)) {	1	n−1	n−1

drivers.remove();	1	0	0
return true;	1	0	0
}	0	–	0
drivers.findNext();	1	n–1	n–1
}	0	–	0
if (drivers.retrieve().getVehiclePlate().equals(vehiclePlate)) {	1	1	1
drivers.remove();	1	1	1
return true;	1	1	1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**3n + 4**
Big-O	–	–	**O (n)**

10. int removeByName (String fullName)

Statement	S/E	Frequency	Total
public int removeByName (String fullName) {	0	–	0
if (drivers.empty())	1	1	1
return 0;	1	0	0
int count = 0;	1	1	1
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n–1	n–1
drivers.remove();	1	n–1	n–1
count++;	1	n–1	n–1
} else {	0	–	0
drivers.findNext();	1	0	0
}	0	–	0
}	0	–	0
if (drivers.retrieve().getName().equalsIgnoreCase(fullName)) {	1	1	1
drivers.remove();	1	1	1
count++;	1	1	1
}	0	–	0
return count;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O (n)**

11. int removeByVehicleType (VehicleType vehicleType)

Statement	S/E	Frequency	Total
public int removeByVehicleType(VehicleType vehicleType) {	0	–	0
if (drivers.empty())	1	1	1

return 0;	1	0	0
int count = 0;	1	1	1
drivers.findFirst();	1	1	1
while (!drivers.last()) {	1	n	n
if (drivers.retrieve().getVehicleType() == vehicleType) {	1	n−1	n−1
drivers.remove();	1	n−1	n−1
count++;	1	n−1	n−1
} else {	0	–	0
drivers.findNext();	1	0	0
}	0	–	0
}	0	–	0
if (drivers.retrieve().getVehicleType() == vehicleType) {	1	1	1
drivers.remove();	1	1	1
count++;	1	1	1
}	0	–	0
return count;	1	1	1
}	0	–	0
Total	–	–	**4n + 4**
Big-O	–	–	**O(n)**

12. int size ()

Statement	S/E	Frequency	Total
public int size() {	0	–	0
return drivers.size();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- RideList

1. public RideList ()

Statement	S/E	Frequency	Total
public RideList() {	0	–	0
rides = new LinkedList<>();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

2. private boolean hasRide (int id)

Statement	S/E	Frequency	Total
private boolean hasRide (int id) {	0	–	0
if (rides.empty()) return false;	1	1	1
rides.findFirst();	1	1	1
while (true) {	1	n	n

if (rides.retrieve().getRideId() == id) return true;	1	n	n
if (rides.last()) break;	1	n	n
rides.findNext();	1	n−1	n−1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**3n + 2**
Big-O	–	–	**O (n)**

3. public boolean addRide (IRide ride)

Statement	S/E	Frequency	Total
public boolean addRide(IRide ride) {	0	–	0
if (ride == null || hasRide(ride.getRideId())) return false;	1	1 (includes hasRide: O(n))	1 (includes hasRide: O(n))
if (rides.empty()) {	1	1	1
rides.insert(ride);	1	0	0
return true;	1	0	0
}	0	–	0
rides.findFirst();	1	1	1
if (rides.retrieve().compareTo(ride) > 0) {	1	1	1
rides.insertFirst(ride);	1	0	0
return true;	1	0	0
}	0	–	0
while (!rides.last()) {	1	n−1	n−1
rides.findNext();	1	n−1	n−1
if (rides.retrieve().compareTo(ride) > 0) {	1	n−1	n−1
rides.findPrevious();	1	0	0
rides.insert(ride);	1	0	0
return true;	1	0	0
}	0	–	0
}	0	–	0
rides.insert(ride);	1	1	1
return true;	1	1	1
}	0	–	0
Total	–	–	**O(n) + O(n) = O(n)**
Big-O	–	–	**O (n)**

4. public boolean removeRideById (int rideId)

Statement	S/E	Frequency	Total
public boolean removeRideById(int rideId) {	0	–	0
if (rides.empty()) return false;	1	1	1
rides.findFirst();	1	1	1
while (true) {	1	n	n

if (rides.retrieve().getRideId() == rideId) {	1	n	n
rides.remove();	1	0	0
return true;	1	0	0
}	0	–	0
if (rides.last()) break;	1	n	n
rides.findNext();	1	n–1	n–1
}	0	–	0
return false;	1	1	1
}	0	–	0
Total	–	–	**3n + 2**
Big-O	–	–	**O (n)**

5. public LinkedList<IRide> getAllAlphabetically ()

Statement	S/E	Frequency	Total
public LinkedList<IRide> getAllAlphabetically() {	0	–	0
return rides;	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

6. public LinkedList<IRide> findByPickupLocation (String pickupLocation)

Statement	S/E	Frequency	Total
public LinkedList<IRide> findByPickupLocation(String pickupLocation) {	0	–	0
LinkedList<IRide> result = new LinkedList<>();	1	1	1
if (rides.empty()) return result;	1	1	1
rides.findFirst();	1	1	1
while (true) {	1	n	n
if (rides.retrieve().getPickupLocation().equalsIgnoreCase(pickupLocation)) result.insert(rides.retrieve());	1	n	n
if (rides.last()) break;	1	n	n
rides.findNext();	1	n–1	n–1
}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**4n + 3**
Big-O	–	–	**O (n)**

7. public LinkedList<IRide> findByRiderName (String riderFullName)

Statement	S/E	Frequency	Total
public LinkedList<IRide> findByRiderName(String riderFullName) {	0	–	0
LinkedList<IRide> result = new LinkedList<>();	1	1	1
if (rides.empty()) return result;	1	1	1
rides.findFirst();	1	1	1
while (!rides.last()) {	1	n–1	n–1
IRide ride = rides.retrieve();	1	n–1	n–1

if (ride instanceof IPrivateRide) {	1	n	n
if (((IPrivateRide) ride).getRider().getName().equalsIgnoreCase(riderFullName)) {	1	0	0
result.insert(ride);	1	0	0
}	0	–	0
}	0	–	0
else if (ride instanceof ISharedRide) {	1	n	n
LinkedList<IRider> participants = ((ISharedRide) ride).getParticipants();	1	n	n
boolean found = false;	1	n	n
if (!participants.empty()) {	1	n	n
participants.findFirst();	1	n	n
while (!participants.last()) {	1	(n–1)(m–1) + (m–1)	(n–1)(m–1) + (m–1)
if (participants.retrieve().getName().equalsIgnoreCase(riderFullName)) {	1	n(m–1)	n(m–1)
found = true;	1	0	0
break;	1	0	0
}	0	–	0
participants.findNext();	1	n(m–1)	n(m–1)
}	0	–	0
if (!found && participants.retrieve().getName().equalsIgnoreCase(riderFullName)) {	1	n	n
found = true;	1	n	n
}	0	–	0
if (found) result.insert(ride);	1	n	n
}	0	–	0
}	0	–	0
rides.findNext();	1	n–1	n–1
}	0	–	0
IRide ride = rides.retrieve();	1	1	1
if (ride instanceof IPrivateRide) {	1	n	n
if (((IPrivateRide) ride).getRider().getName().equalsIgnoreCase(riderFullName)) {	1	0	0
result.insert(ride);	1	0	0
}	0	–	0
}	0	–	0
else if (ride instanceof ISharedRide) {	1	n	n
LinkedList<IRider> participants = ((ISharedRide) ride).getParticipants();	1	n	n
boolean found = false;	1	n	n
if (!participants.empty()) {	1	n	n
participants.findFirst();	1	n	n
while (!participants.last()) {	1	(n–1)(m–1) + (m–1)	(n–1)(m–1) + (m–1)
if (participants.retrieve().getName().equalsIgnoreCase(riderFullName)) {	1	n(m–1)	n(m–1)
found = true;	1	0	0
break;	1	0	0
}	0	–	0

participants.findNext();	1	n(m−1)	n(m−1)
}	0	–	0
if (!found && participants.retrieve().getName().equalsIgnoreCase(riderFullName)) {	1	n	n
found = true;	1	n	n
}	0	–	0
if (found) result.insert(ride);	1	n	n
}	0	–	0
}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**O (nm + n)**
Big-O	–	–	**O (nm)**

8. public int size()

Statement	S/E	Frequency	Total
public int size() {	0	–	0
return rides.size();	1	1	1
}	0	–	0
Total	–	–	**1**
Big-O	–	–	**O (1)**

- RideSharingSystem

1. RideSharingSystem ()

Statement	S/E	Frequency	Total
public RideSharingSystem () {	0	–	0
this.riderList = new RiderList();	1	1	1
this.driverList = new DriverList();	1	1	1
this.rideList = new RideList();	1	1	1
this.nextRideId = 1;	1	1	1
}	0	–	0
Total	–	–	**4**
Big-O	–	–	**O (1)**

2. parseDateTime ()

Statement	S/E	Frequency	Total
private IDateTime parseDateTime (String dateTimeStr) {	0	–	0
String[] parts = dateTimeStr.trim().split(" ");	1	1	1
String[] dateParts = parts[0].split("/");	1	1	1
String[] timeParts = parts[1].split(":");	1	1	1
int month = Integer.parseInt(dateParts[0]);	1	1	1
int day = Integer.parseInt(dateParts[1]);	1	1	1
int year = Integer.parseInt(dateParts[2]);	1	1	1
int hour = Integer.parseInt(timeParts[0]);	1	1	1
int minute = Integer.parseInt(timeParts[1]);	1	1	1
return new DateTime(year, month, day, hour, minute);	1	1	1

}	0	–	0
Total	–	–	**9**
Big-O	–	–	**O (1)**

3. loadRidersFromCSV ()

Statement	S/E	Frequency	Total
public boolean loadRidersFromCSV(String ridersFilePath) {	0	–	0
try (BufferedReader br = new BufferedReader(new FileReader(ridersFilePath))) {	1	1	1
String line;	1	1	1
while ((line = br.readLine()) != null) {	1	f+1	f+1
line = line.trim();	1	f	f
if (line.isEmpty()) continue;	1	f	f
String[] tokens = line.split(",");	1	f	f
int id = Integer.parseInt(tokens[0].trim());	1	f	f
String name = tokens[1].trim();	1	f	f
String email = tokens[2].trim();	1	f	f
String phone = tokens[3].trim();	1	f	f
String city = tokens[4].trim();	1	f	f
IRider rider = new Rider(id, name, phone, email, city);	1	f	f
addRider(rider);	1	f	f × O(n)
}	0	–	0
return true;	1	≤ 1	≤ 1
} catch (IOException | IllegalArgumentException e) {	0	–	0
return false;	1	≤ 1	≤ 1
}	0	–	0
}	0	–	0
Total	–	–	**T = O(fn)**
Big-O	–	–	**O (fn)**

4. loadDriversFromCSV ()

Statement	S/E	Frequency	Total
public boolean loadDriversFromCSV(String driversFilePath) {	0	–	0
try (BufferedReader br = new BufferedReader(new FileReader(driversFilePath))) {	1	1	1
String line;	1	1	1
while ((line = br.readLine()) != null) {	1	f+1	f+1
line = line.trim();	1	f	f
if (line.isEmpty()) continue;	1	f	f
String[] tokens = line.split(",");	1	f	f
int id = Integer.parseInt(tokens[0].trim());	1	f	f
String name = tokens[1].trim();	1	f	f
String phone = tokens[2].trim();	1	f	f
String plate = tokens[3].trim();	1	f	f
VehicleType vehicleType = VehicleType.valueOf(tokens[4].trim());	1	f	f
IDriver driver = new Driver(id, name, phone, plate, vehicleType);	1	f	f
addDriver(driver);	1	f	f × O(d)
}	0	–	0
return true;	1	≤ 1	≤ 1
} catch (IOException | IllegalArgumentException e) {	0	–	0
return false;	1	≤ 1	≤ 1

}	0	–	0
}	0	–	0
Total	–	–	**T = O(fd)**
Big-O	–	–	**O (fd)**

5. loadRidesFromCSV ()

Statement	S/E	Frequency	Total
public boolean loadRidesFromCSV(String ridesFilePath) {	0	–	0
try (BufferedReader br = new BufferedReader(new FileReader(ridesFilePath))) {	1	1	1
String line;	1	1	1
while ((line = br.readLine()) != null) {	1	f+1	f+1
line = line.trim();	1	f	f
if (line.isEmpty()) continue;	1	f	f
String[] tokens = line.split(",");	1	f	f
String rideType = tokens[0].trim();	1	f	f
String pickupLoc = tokens[1].trim();	1	f	f
IDateTime pickupTime = parseDateTime(tokens[2]);	1	f	f
IDateTime dropoffTime = parseDateTime(tokens[3]);	1	f	f
String dropoffLoc = tokens[4].trim();	1	f	f
int driverId = Integer.parseInt(tokens[5].trim());	1	f	f
if (rideType.equalsIgnoreCase("PRIVATE")) {	1	f	f
int riderId = Integer.parseInt(tokens[6].trim());	1	f	f
schedulePrivateRide(pickupLoc, pickupTime, dropoffTime, dropoffLoc, riderId, driverId);	1	f	f × O(n+d+h+r)
} else if (rideType.equalsIgnoreCase("SHARED")) {	0	–	0
String[] idStrs = tokens[6].trim().split(";");	1	f	f
int[] riderIds = new int[idStrs.length];	1	f	f
for (int i = 0; i < idStrs.length; i++) {	1	f(k+1)	f(k+1)
riderIds[i] = Integer.parseInt(idStrs[i].trim());	1	fk	fk
}	0	–	0
scheduleSharedRide(pickupLoc, pickupTime, dropoffTime, dropoffLoc, riderIds, driverId);	1	f	f × O(d+h+k²+kn+kh+kp+r)
}	0	–	0
}	0	–	0
return true;	1	≤ 1	≤ 1
} catch (IOException | IllegalArgumentException e) {	0	–	0
return false;	1	≤ 1	≤ 1
}	0	–	0
}	0	–	0
Total	–	–	**T = O(f(r + d + kn + k² + kh + kp))**
Big-O	–	–	**O (f(r + d + kn + k² + kh + kp))**

6. addRider ()

Statement	S/E	Frequency	Total
public boolean addRider(IRider rider) {	0	–	0
if (rider == null) return false;	1	1	1
return riderList.add(rider);	1	1	1 × O(n)

}	0	–	0
Total	–	–	T = O(n)
Big-O	**–**	**–**	**O (n)**

7. addDriver ()

Statement	S/E	Frequency	Total
public boolean addDriver(IDriver driver) {	0	–	0
if (driver == null) return false;	1	1	1
return driverList.add(driver);	1	1	1 × O(d)
}	0	–	0
Total	–	–	**T = O(d)**
Big-O	–	–	**O (d)**

8. searchRiderById ()

Statement	S/E	Frequency	Total
public IRider searchRiderById (int riderId) {	0	–	0
return riderList.findById(riderId);	1	1	1 × O(n)
}	0	–	0
Total	–	–	**T = O(n)**
Big-O	–	–	**O (n)**

9. searchRiderByEmail ()

Statement	S/E	Frequency	Total
public IRider searchRiderByEmail(String email) {	0	–	0
return riderList.findByEmail(email);	1	1	1 × O(n)
}	0	–	0
Total	–	–	**T = O(n)**
Big-O	–	–	**O (n)**

10. searchRidersByName ()

Statement	S/E	Frequency	Total
public LinkedList<IRider> searchRidersByName(String fullName) {	0	–	0
return riderList.findByName(fullName);	1	1	1 × O(n)
}	0	–	0
Total	–	–	**T = O(n)**
Big-O	–	–	**O (n)**

11. searchRidersByHomeCity ()

Statement	S/E	Frequency	Total
public LinkedList<IRider> searchRidersByHomeCity(String homeCity) {	0	–	0
return riderList.findByHomeCity(homeCity);	1	1	1 × O(n)
}	0	–	0
Total	–	–	**T = O (n)**
Big-O	–	–	**O (n)**

12. getAllRiders ()

Statement	S/E	Frequency	Total

public LinkedList<IRider> getAllRiders() {	0	–	0
return riderList.getAll();	1	1	1
}	0	–	0
Total	–	–	**T = O(1)**
Big-O	–	–	**O (1)**

13. searchDriverById ()

Statement	S/E	Frequency	Total
public IDriver searchDriverById(int driverId) {	0	–	0
return driverList.findById(driverId);	1	1	1 × O(d)
}	0	–	0
Total	–	–	**T = O(d)**
Big-O	–	–	**O (d)**

14. searchDriverByVehiclePlate ()

Statement	S/E	Frequency	Total
public IDriver searchDriverByVehiclePlate(String vehiclePlate) {	0	–	0
return driverList.findByVehiclePlate(vehiclePlate);	1	1	1 × O(d)
}	0	–	0
Total	–	–	**T = O(d)**
Big-O	–	–	**O (d)**

15. searchDriversByVehicleType ()

Statement	S/E	Frequency	Total
public LinkedList<IDriver> searchDriversByVehicleType(VehicleType vehicleType) {	0	–	0
return driverList.findByVehicleType(vehicleType);	1	1	1 × O(d)
}	0	–	0
Total	–	–	**T = O(d)**
Big-O	–	–	**O(d)**

16. getAllDrivers()

Statement	S/E	Frequency	Total
public LinkedList<IDriver> getAllDrivers() {	0	–	0
return driverList.getAll();	1	1	1
}	0	–	0
Total	–	–	**T = O (1)**
Big-O	–	–	**O (1)**

17. removeRider()

Statement	S/E	Frequency	Total
public boolean removeRider (int riderId) {	0	–	0
IRider rider = riderList.findById(riderId);	1	1	1 × O(n)
if (rider == null) return false;	1	1	1
LinkedList<IRide> history = rider.getRideHistory();	1	1	1
while (!history.empty()) {	1	h+1	h+1
history.findFirst();	1	h	h
IRide ride = history.retrieve();	1	h	h
history.remove();	1	h	h

if (ride instanceof IPrivateRide) {	1	h	h
deleteRide(ride);	1	h	h × O(r+ph+p)
} else if (ride instanceof ISharedRide) {	0	–	0
ISharedRide sharedRide = (ISharedRide) ride;	1	h	h
sharedRide.removeParticipantById(riderId);	1	h	h × O(p)
if (sharedRide.isEmpty())	1	h	h
deleteRide(ride);	1	h	h × O(r+ph+p)
}	0	–	0
}	0	–	0
riderList.removeById(riderId);	1	1	1 × O(n)
return true;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(n + h(r + h + p))**
Big-O	–	–	**O (n + h(r + h + p))**

18. removeDriver()

Statement	S/E	Frequency	Total
public boolean removeDriver (int driverId) {	0	–	0
IDriver driver = driverList.findById(driverId);	1	1	1 × O(d)
if (driver == null) return false;	1	1	1
LinkedList<IRide> history = driver.getRideHistory();	1	1	1
while (!history.empty()) {	1	h+1	h+1
history.findFirst();	1	h	h
IRide ride = history.retrieve();	1	h	h
history.remove();	1	h	h
deleteRide (ride);	1	h	h × O(r+ph+p)
}	0	–	0
driverList.removeById(driverId);	1	1	1 × O(d)
return true;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(d + h(r + h + p))**
Big-O	–	–	**O (d + h(r + h + p))**

19. schedulePrivateRide ()

Statement	S/E	Frequency	Total
public boolean schedulePrivateRide (String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, int riderId, int driverId) {	0	–	0
if (!isValidRideInfo(pickupLocation, pickupTime, dropoffTime, dropoffLocation)) return false;	1	1	1
IRider rider = riderList.findById(riderId);	1	1	1 × O(n)
IDriver driver = driverList.findById(driverId);	1	1	1 × O(d)
if (rider == null || driver == null) return false;	1	1	1
if (hasConflict(rider.getRideHistory(), pickupTime, dropoffTime)) return false;	1	1	1 × O(h)
if (hasConflict(driver.getRideHistory(), pickupTime, dropoffTime)) return false;	1	1	1 × O(h)
IPrivateRide ride = new PrivateRide(nextRideId++, pickupTime, dropoffTime, pickupLocation, dropoffLocation, driver, rider);	1	1	1
rideList.addRide(ride);	1	1	1 × O(r)
rider.getRideHistory().insert(ride);	1	1	1

driver.getRideHistory().insert(ride);	1	1	1
return true;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(n + d + h + r)**
Big-O	–	–	**O (n + d + h + r)**

20. scheduleSharedRide()

Statement	S/E	Frequency	Total
public boolean scheduleSharedRide (String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, int[] riderIds, int driverId) {	0	–	0
if (!isValidRideInfo(pickupLocation, pickupTime, dropoffTime, dropoffLocation)) return false;	1	1	1
if (riderIds == null || riderIds.length == 0) return false;	1	1	1
IDriver driver = driverList.findById(driverId);	1	1	1 × O(d)
if (driver == null) return false;	1	1	1
if (hasConflict(driver.getRideHistory(), pickupTime, dropoffTime)) return false;	1	1	1 × O(h)
IRider[] riders = new IRider[riderIds.length];	1	1	1
for (int i = 0; i < riderIds.length; i++) {	1	k+1	k+1
for (int j = 0; j < i; j++) {	1	k(k+1)/2	k(k+1)/2
if (riderIds[j] == riderIds[i]) return false;	1	k(k−1)/2	k(k−1)/2
}	0	–	0
riders[i] = riderList.findById(riderIds[i]);	1	k	k × O(n)
if (riders[i] == null) return false;	1	k	k
if (hasConflict(riders[i].getRideHistory(), pickupTime, dropoffTime)) return false;	1	k	k × O(h)
}	0	–	0
ISharedRide ride = new SharedRide(nextRideId++, pickupTime, dropoffTime, pickupLocation, dropoffLocation, driver);	1	1	1
for (int i = 0; i < riders.length; i++) {	1	k+1	k+1
ride.addParticipant(riders[i]);	1	k	k × O(p)
riders[i].getRideHistory().insert(ride);	1	k	k
}	0	–	0
rideList.addRide(ride);	1	1	1 × O(r)
driver.getRideHistory().insert(ride);	1	1	1
return true;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(d + h + k^2 + kn + kh + kp + r)**
Big-O	–	–	**O(d + h + k^2 + kn + kh + kp + r)**

21. searchRidesByPickupLocation ()

Statement	S/E	Frequency	Total
public LinkedList<IRide> searchRidesByPickupLocation(String pickupLocation) {	0	–	0
return rideList.findByPickupLocation(pickupLocation);	1	1	1 × O(r)
}	0	–	0
Total	–	–	**T = O(r)**
Big-O	–	–	**O(r)**

22. searchRidesByRiderName()

Statement	S/E	Frequency	Total
public LinkedList<IRide> searchRidesByRiderName(String riderName) {	0	–	0
return rideList.findByRiderName(riderName);	1	1	1 × O(rp)
}	0	–	0
Total	–	–	**T = O(rp)**
Big-O	–	–	**O (rp)**

23. getSharedRideParticipants ()

Statement	S/E	Frequency	Total
public LinkedList<IRider> getSharedRideParticipants(String pickupLocation) {	0	–	0
LinkedList<IRider> result = new LinkedList<IRider>();	1	1	1
LinkedList<IRide> matches = rideList.findByPickupLocation(pickupLocation);	1	1	1 × O(r)
if (matches.empty()) return result;	1	1	1
matches.findFirst();	1	1	1
while (true) {	1	r+1	r+1
if (matches.retrieve() instanceof ISharedRide) {	1	r	r
LinkedList<IRider> participants = ((ISharedRide) matches.retrieve()).getParticipants();	1	r	r
if (!participants.empty()) {	1	r	r
participants.findFirst();	1	r	r
while (true) {	1	r(p+1)	r(p+1)
if (!containsRider(result, participants.retrieve().getId()))	1	rp	rp × O(p)
result.insert(participants.retrieve());	1	rp	rp
if (participants.last()) break;	1	rp	rp
participants.findNext();	1	rp	rp
}	0	–	0
}	0	–	0
}	0	–	0
if (matches.last()) break;	1	r	r
matches.findNext();	1	r	r
}	0	–	0
return result;	1	1	1
}	0	–	0
Total	–	–	**T = O(r + rp + (rp)²)**
Big-O	–	–	**O (r + rp + (rp)²)**

24. getAllRidesAlphabetically()

Statement	S/E	Frequency	Total
public LinkedList<IRide> getAllRidesAlphabetically() {	0	–	0
return rideList.getAllAlphabetically();	1	1	1
}	0	–	0
Total	–	–	**T = O(1)**
Big-O	–	–	**O(1)**

25. isValidRideInfo ()

Statement	S/E	Frequency	Total
private boolean isValidRideInfo(String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation) {	0	–	0

if (pickupLocation == null || pickupLocation.trim().isEmpty()) return false;	1	1	1
if (dropoffLocation == null || dropoffLocation.trim().isEmpty()) return false;	1	1	1
if (pickupTime == null || dropoffTime == null) return false;	1	1	1
return pickupTime.compareTo(dropoffTime) < 0;	1	1	1
}	0	–	0
Total	–	–	**4**
Big-O	–	–	**O (1)**

26. hasConflict ()

Statement	S/E	Frequency	Total
private boolean hasConflict(LinkedList<IRide> history, IDateTime pickupTime, IDateTime dropoffTime) {	0	–	0
if (history.empty()) return false;	1	1	1
history.findFirst();	1	1	1
while (true) {	1	h	h
IRide ride = history.retrieve();	1	h	h
if (pickupTime.compareTo(ride.getDropoffTime()) < 0 && ride.getPickupTime().compareTo(dropoffTime) < 0)	1	h	h
return true;	1	≤ h	≤ h
if (history.last()) break;	1	h	h
history.findNext();	1	h	h
}	0	–	0
return false;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(h)**
Big-O	–	–	**O (h)**

27. deleteRide ()

Statement	S/E	Frequency	Total
private void deleteRide (IRide ride) {	0	–	0
int rideId = ride.getRideId();	1	1	1
rideList.removeRideById(rideId);	1	1	1 × O(r)
removeFromHistory (ride.getDriver().getRideHistory(), rideId);	1	1	1
if (ride instanceof IPrivateRide) {	1	1	1
removeFromHistory(((IPrivateRide) ride).getRider().getRideHistory(), rideId);	1	1	1 × O(h)
} else if (ride instanceof ISharedRide) {	0	–	0
LinkedList<IRider> participants = ((ISharedRide) ride).getParticipants();	1	1	1
if (participants.empty()) return;	1	1	1
participants.findFirst();	1	1	1
while (true) {	1	p+1	p+1
removeFromHistory(participants.retrieve().getRideHistory(), rideId);	1	p	p × O(h)
if (participants.last()) break;	1	p	p
participants.findNext();	1	p	p
}	0	–	0
}	0	–	0
}	0	–	0
Total	–	–	**T = O(r + ph + p)**
Big-O	–	–	**O(r + ph + p)**

28. removeFromHistory ()

Statement	S/E	Frequency	Total
private void removeFromHistory (LinkedList<IRide> history, int rideId) {	0	–	0
if (history.empty()) return;	1	1	1
history.findFirst();	1	1	1
while (true) {	1	h	h
if (history.retrieve().getRideId() == rideId) {	1	h	h
history.remove();	1	h	h
return;	1	≤ h	≤ h
}	0	–	0
if (history.last()) break;	1	h	h
history.findNext();	1	h	h
}	0	–	0
}	0	–	0
Total	–	–	**T = O(h)**
Big-O	–	–	**O (h)**

29. containsRider()

Statement	S/E	Frequency	Total
private boolean containsRider(LinkedList<IRider> riders, int riderId) {	0	–	0
if (riders.empty()) return false;	1	1	1
riders.findFirst();	1	1	1
while (true) {	1	p	p
if (riders.retrieve().getId() == riderId) return true;	1	p	p
if (riders.last()) break;	1	p	p
riders.findNext();	1	p	p
}	0	–	0
return false;	1	≤ 1	≤ 1
}	0	–	0
Total	–	–	**T = O(p)**
Big-O	–	–	**O(p)**

- Main

1. main ()

Statement	S/E	Frequency	Total
public static void main (String[] args)	0	–	0
{	0	–	0
system.loadDriversFromCSV("csv\\drivers_30.csv");	1	1	1
system.loadRidersFromCSV("csv\\riders_100.csv");	1	1	1
system.loadRidesFromCSV("csv\\rides_40.csv");	1	1	1
while (true) {	1	m + 1	m + 1
printMenu();	1	m	m
try {	0	–	0
int choose = readInt("➜ Enter your choice (0–12): ", "[!] Invalid input: Please enter a valid integer (0–12).");	1	m	m
switch (choose) {	1	m	m
case 0 -> {	0	–	0

System.out.println("Thank you for using the Ride-Sharing System. Goodbye!");	1	m	m
return;	1	m	m
}	0	–	0
case 1 -> {	0	–	0
System.out.println("\n═══════════════════ [1] ALL REGISTERED RIDERS ═══════════════════");	1	m	m
printList(system.getAllRiders(), "No riders found. There are currently no registered riders in the system.");	1	m	m
}	0	–	0
case 2 -> {	0	–	0
String city = readLine("Enter home city to search: ");	1	m	m
System.out.println("\n═══════════════════ [2] SEARCH RIDERS BY HOME CITY ═══════════════════");	1	m	m
printList(system.searchRidersByHomeCity(city), "No riders found in the specified home city.");	1	m	m
}	0	–	0
case 3 -> {	0	–	0
String name = readLine("Enter rider name to search: ");	1	m	m
System.out.println("\n═══════════════════ [3] SEARCH RIDERS BY NAME ═══════════════════");	1	m	m
printList(system.searchRidersByName(name), "No riders found matching the specified name.");	1	m	m
}	0	–	0
case 4 -> {	0	–	0
System.out.println("\n═══════════════════ [4] ADD NEW RIDER ═══════════════════");	1	m	m
addRider();	1	m	m
}	0	–	0
case 5 -> {	0	–	0
System.out.println("\n═══════════════════ [5] ALL REGISTERED DRIVERS ═══════════════════");	1	m	m
printList(system.getAllDrivers(), "No drivers found. There are currently no registered drivers in the system.");	1	m	m
}	0	–	0
case 6 -> {	0	–	0
VehicleType vehicleType = readVehicleType();	1	m	m
System.out.println("\n═══════════════════ [6] SEARCH DRIVERS BY VEHICLE TYPE ═══════════════════");	1	m	m
printList(system.searchDriversByVehicleType(vehicleType), "No drivers found with the specified vehicle type.");	1	m	m
}	0	–	0
case 7 -> {	0	–	0
System.out.println("\n═══════════════════ [7] ADD NEW DRIVER ═══════════════════");	1	m	m
addDriver();	1	m	m
}	0	–	0
case 8 -> {	0	–	0
System.out.println("\n═══════════════════ [8] ALL RIDES BY PICKUP LOCATION ═══════════════════");	1	m	m
printList(system.getAllRidesAlphabetically(), "No rides found. There are currently no scheduled rides in the system.");	1	m	m
}	0	–	0
case 9 -> {	0	–	0

String pickupLocation = readLine("Enter pickup location to search:");	1	m	m
System.out.println("\n═══════════════════ [9] SEARCH RIDES BY PICKUP LOCATION ═══════════════════");	1	m	m
printList(system.searchRidesByPickupLocation(pickupLocation),"No rides found matching the specified pickup location.");	1	m	m
}	0	–	0
case 10 -> {	0	–	0
String name = readLine("Enter rider name to search: ");	1	m	m
System.out.println("\n═══════════════════ [10] SEARCH RIDES BY RIDER NAME ═══════════════════");	1	m	m
printList(system.searchRidesByRiderName(name),"No rides found for the specified rider name.");	1	m	m
}	0	–	0
case 11 -> {	0	–	0
String pickupLocation = readLine("Enter shared ride pickup location: ");	1	m	m
System.out.println("\n═══════════════════ [11] SHARED RIDE PARTICIPANTS BY PICKUP LOCATION ═══════════════════");	1	m	m
printList(system.getSharedRideParticipants(pickupLocation),"\"No participants found for shared rides at the specified pickup location.\"");	1	m	m
}	0	–	0
case 12 -> {	0	–	0
System.out.println("\n═══════════════════ [12] SCHEDULE A NEW RIDE ═══════════════════");	1	m	m
addRide();	1	m	m
}	0	–	0
default -> System.out.println(" [!] Invalid choice: Please enter a number between 0 and 12.");	0	–	0
}	0	–	0
} catch (Exception e) {	0	–	0
System.out.println(e.getMessage());	1	m	m
}	0	–	0
}	0	–	0
}	0	–	0
Total	–	–	3 + m + 1 + 36(m)
Big-O	–	–	O(m · (n + p + k))

2. printMenu ()

Statement	S/E	Frequency	Total
private static void printMenu ()	0	–	0
{	0	–	0
System.out.println();	1	1	1
System.out.println("╔═══╗");	1	1	1
System.out.println("║ RIDE-SHARING MANAGEMENT SYSTEM ║");	1	1	1
System.out.println("║ Main Control Panel ║");	1	1	1
System.out.println("╠═══╣");	1	1	1
System.out.println("║ RIDER MANAGEMENT ║");	1	1	1
System.out.println("╟───╢");	1	1	1
System.out.println("║ [1] List all riders ║");	1	1	1

System.out.println("\u2551 [2] Search riders by home city \u2551");	1	1	1
System.out.println("\u2551 [3] Search riders by name \u2551");	1	1	1
System.out.println("\u2551 [4] Add a new rider \u2551");	1	1	1
System.out.println("\u2551\u2500\u2500\u2500

4. addRider ()

Statement	S/E	Frequency	Total
private static void addRider ()	0	–	0
{	0	–	0
int id = readInt("Enter rider ID: ", "Please enter a valid whole number.");	1	1	1
String name = readLine("Enter full name: ");	1	1	1
String email = readLine("Enter email: ");	1	1	1
String phone = readLine("Enter phone number (10 digits): ");	1	1	1
String city = readLine("Enter home city: ");	1	1	1
try {	0	–	0
if (system.addRider(new Rider(id, name, phone, email, city)))	1	1	1
System.out.println("[+] Rider added successfully.");	1	1	1
else	1	1	1
System.out.println("[!] Failed to add rider: A rider with this ID or email already exists.");	1	1	1
} catch (IllegalArgumentException e) {	0	–	0
System.out.println("[!] Failed to add rider: " + e.getMessage());	1	1	1
}	0	–	0
}	0	–	0
Total	–	–	10
Big-O	–	–	O(n + k)

5. addDriver ()

Statement	S/E	Frequency	Total
private static void addDriver ()	0	–	0
{	0	–	0
int id = readInt("Enter driver ID: ", "Please enter a valid whole number.");	1	1	1
String name = readLine("Enter full name: ");	1	1	1
String phone = readLine("Enter phone number (10 digits): ");	1	1	1
String plate = readLine("Enter vehicle plate (e.g., ABC1234): ");	1	1	1
VehicleType vehicleType = readVehicleType();	1	1	1
try {	0	–	0
if (system.addDriver(new Driver(id, name, phone, plate, vehicleType)))	1	1	1
System.out.println("[+] Driver added successfully.");	1	1	1
else	1	1	1
System.out.println("[!] Failed to add driver: a driver with this ID or vehicle plate already exists.");	1	1	1
} catch (IllegalArgumentException e) {	0	–	0
System.out.println("[!] Failed to add driver: " + e.getMessage());	1	1	1
}	0	–	0
}	0	–	0
Total	–	–	10
Big-O	–	–	O(n + k)

6. addRide ()

Statement	S/E	Frequency	Total
private static void addRide ()	0	–	0
{	0	–	0
System.out.println("Select the ride type: ");	1	1	1
System.out.println("[1] Private Ride");	1	1	1

System.out.println("[2] Shared Ride");	1	1	1
int rideType;	1	1	1
while (true) {	1	k + 1	k + 1
rideType = readInt("Enter ride type (1–2): ", "[!] Invalid input. Please enter a valid integer.");	1	k	k
if (rideType == 1 || rideType == 2) break;	1	k	k
System.out.println("[!] Invalid ride type. Please enter 1 for Private or 2 for Shared.");	1	1	1
}	0	–	0
if (rideType == 1)	1	1	1
System.out.println("\n———————————— PRIVATE RIDE DETAILS ————————————");	1	1	1
else	1	1	1
System.out.println("\n———————————— SHARED RIDE DETAILS ————————————");	1	1	1
String pickupLocation = readLine("Enter pickup location: ");	1	1	1
IDateTime pickupTime = readDateTime("Enter pickup date and time (MM/DD/YYYY HH:MM): ");	1	1	1
String dropOffLocation = readLine("Enter drop-off location: ");	1	1	1
IDateTime dropOffTime = readDateTime("Enter drop-off date and time (MM/DD/YYYY HH:MM): ");	1	1	1
int driverId = readInt("Enter driver ID: ", "Please enter a valid whole number");	1	1	1
if (rideType == 1) {	1	1	1
int riderId = readInt("Enter rider ID: ", "Please enter a valid whole number");	1	1	1
try {	0	–	0
if (system.schedulePrivateRide(pickupLocation,pickupTime,dropOffTime,dropOffLocation,riderId,driverId))	1	1	1
System.out.println("[+] Private ride scheduled successfully.");	1	1	1
else	1	1	1
System.out.println("[!] Failed to schedule private ride. Please verify the rider, driver, and ride details.");	1	1	1
} catch (IllegalArgumentException e) {	0	–	0
System.out.println("[!] Failed to schedule private ride: " + e.getMessage());	1	1	1
}	0	–	0
} else {	0	–	0
int numberOfParticipants;	1	1	1
while (true) {	1	k + 1	k + 1
numberOfParticipants = readInt("Enter the number of participants: ","Please enter a valid whole number");	1	k	k
if (numberOfParticipants > 0) break;	1	1	1
System.out.println("[!] Invalid number of participants. Please enter a positive integer.");	1	1	1
}	0	–	0
int[] riderIds = new int[numberOfParticipants];	1	1	1
for (int i = 0; i < numberOfParticipants; i++) {	1	p + 1	p + 1
riderIds[i] = readInt(String.format("Enter rider ID for participant (%d): ", i + 1),"Please enter a valid whole number");	1	p	p
}	0	–	0
try {	0	–	0
if (system.scheduleSharedRide(pickupLocation,pickupTime,dropOffTime,dropOffLocation,riderIds,driverId))	1	1	1
System.out.println("[+] Shared ride scheduled successfully.");	1	1	1
else	1	1	1
System.out.println("[!] Failed to schedule shared ride. Please verify the participants, driver, and ride details.");	1	1	1

} catch (IllegalArgumentException e) {	0	–	0
System.out.println("[!] Failed to schedule shared ride: " + e.getMessage());	1	1	1
}	0	–	0
}	0	–	0
}	0	–	0
Total	–	–	30 + 2(k + 1) + 3(k) + p + 1 + p
Big-O	–	–	O(n + p + k)

7. readLine ()

Statement	S/E	Frequency	Total
private static String readLine (String prompt)	0	–	0
{	0	–	0
System.out.print(prompt);	1	1	1
try {	0	–	0
String line = input.readLine();	1	1	1
if (line == null) {	1	1	1
System.out.println();	1	1	1
System.out.println("Input ended. Goodbye!");	1	1	1
System.exit(0);	1	1	1
}	0	–	0
return line.trim();	1	1	1
} catch (IOException e) {	0	–	0
System.out.println("Could not read input. Goodbye!");	1	1	1
System.exit(1);	1	1	1
return "";	1	1	1
}	0	–	0
}	0	–	0
Total	–	–	10
Big-O	–	–	O (L)

8. readInt ()

Statement	S/E	Frequency	Total
private static int readInt (String prompt, String invalidMessage)	0	–	0
{	0	–	0
while (true) {	1	k + 1	k + 1
String text = readLine(prompt);	1	k	k
try {	0	–	0
return Integer.parseInt(text);	1	k	k
} catch (NumberFormatException e) {	0	–	0
System.out.println(invalidMessage);	1	k	k
}	0	–	0

}	0	–	0
}	0	–	0
Total	–	–	k + 1 + 3(k)
Big-O	–	–	O(k · L)

9. readVehicleType ()

Statement	S/E	Frequency	Total
private static VehicleType readVehicleType ()	0	–	0
{	0	–	0
while (true) {	1	k + 1	k + 1
String text = readLine("Enter vehicle type (SEDAN, LUXURY_SEDAN, SUV, VAN): ");	1	k	k
try {	0	–	0
return VehicleType.valueOf(text.toUpperCase().replaceAll("\\s+", "_"));	1	k	k
} catch (IllegalArgumentException e) {	0	–	0
System.out.println("[!] Invalid vehicle type. Please choose from the listed options.");	1	k	k
}	0	–	0
}	0	–	0
}	0	–	0
Total	–	–	k + 1 + 3(k)
Big-O	–	–	O(k · L)

10. readDateTime ()

Statement	S/E	Frequency	Total
private static IDateTime readDateTime (String prompt)	0	–	0
{	0	–	0
while (true) {	1	k + 1	k + 1
String text = readLine(prompt);	1	k	k
try {	0	–	0
String[] parts = text.split("\\s+");	1	k	k
if (parts.length != 2) throw new IllegalArgumentException();	1	k	k
String[] date = parts[0].split("/",-1);	1	k	k
String[] time = parts[1].split(":",-1);	1	k	k
if (date.length != 3 || time.length != 2) throw new IllegalArgumentException();	1	k	k
int month = Integer.parseInt(date[0]);	1	k	k
int day = Integer.parseInt(date[1]);	1	k	k
int year = Integer.parseInt(date[2]);	1	k	k
int hour = Integer.parseInt(time[0]);	1	k	k
int minute = Integer.parseInt(time[1]);	1	k	k
return new DateTime(year, month, day, hour, minute);	1	k	k
} catch (IllegalArgumentException e) {	0	–	0
System.out.println("Invalid date/time. Use the format MM/DD/YYYY HH:MM (e.g., 03/15/2026 14:30).");	1	k	k
}	0	–	0
}	0	–	0
}	0	–	0
Total			k + 1 + 13(k)
Big-O			O(k · L)